**SC2002 Object Oriented Design & Programming**
**AY25/26 Semester 2 Group Assignment**

**Turn-Based Combat Arena Game**
**Lab Group: FDAE, Group 6**

**Github Link: https://github.com/wangsengg/SC2002_FDAE_Group6_Turn-Based-Combat-Arena**

**Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment**

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

| Name | Course | Lab Group | Signature/Date |
|---|---|---|---|
| Tan Jun Xi, Javier (U2522831E) | SC2002 | FDAE | Javier 16/04/2026 |
| Toh Wang Seng (U2521717K) | SC2002 | FDAE | Wang Seng 16/04/2026 |
| Tu Jiani (U2523495E) | SC2002 | FDAE | Jiani 16/04/2026 |
| Wang Yifei (U2521069C) | SC2002 | FDAE | Yifei 16/04/2026 |
| Yeu Chong Jun Juninho (U2522725G) | SC2002 | FDAE | Jun 16/04/2026 |

**Contents**

1. Design Considerations 3
1.1 System Design Overview 3
1.2 Understanding the Problem and Requirements 4
1.2.1 Core Features 4
1.2.2 Additional Features 5
1.2.3 Assumptions and Constraints 7
1.3 Application of Object-Oriented Concepts 7
1.4 Application of OOD Principles - SOLID 8
1.4.1 Single Responsibility Principle (SRP) 8
1.4.2 Open-Closed Principle (OCP) 9
1.4.3 Liskov Substitution Principle (LSP) 10
1.4.4 Interface Segregation Principle (ISP) 10
1.4.5 Dependency Inversion Principle (DIP) 11
1.5 Trade-offs and Maintainability Considerations 11
1.6 Design Patterns Used 13
2. Detailed UML Class Diagram 14
3. UML Sequence Diagram 20
4. Testing 22
5. Reflection 26
5.1 Development Challenges 26
5.2 Takeaways 27
5.3 Further Improvements 27

## 1. Design Considerations

### 1.1 System Design Overview

The Turn-Based Combat Arena system is a command-line, turn-based combat game in which a single player battles one or more enemies through a sequence of rounds. The design emphasises modularity and extensibility by separating user interaction, battle coordination, and game-state management into distinct components.

The architecture follows the Boundary-Control-Entity (BCE) model.

- Boundary Layer: responsible for all user interaction
  - GameUI defines the contract for all UI operations
  - GameCLI implements this contract and handles console input validation, menu display, battle-state rendering, and post-game options.
- Control Layer: responsible for coordinating game flow
  - BattleEngine orchestrates the overall battle loop, including round progression, turn execution, backup enemy spawning, and win/loss detection
  - LevelFactory creates the enemy configuration for each difficulty level
  - SpeedTurnOrder determines the order in which combatants act each round
- Entity Layer: responsible for storing and manipulating combat state
  - Combatant, Player, and Enemy represent all battle participants
  - Action, Item, StatusEffect, and SkillCooldown encapsulate specific combat behaviour such as attacking, using items, applying buffs/debuffs, and tracking cooldowns

A major design goal was to support extension without requiring large-scale modification to the existing system. In particular, the architecture allows:

- new enemy types to be added as subclasses of Enemy
- new status effects to be added as subclasses of StatusEffect
- new items and skills to be added without changing the battle loop
- different turn-order algorithms to be introduced by implementing TurnOrderStrategy

Although the overall design follows BCE, some pragmatic coupling remains. BattleEngine currently acts as the central coordinator for many responsibilities, including turn execution, battle-end checking, enemy spawning, and interaction with the UI. This simplifies the overall structure for the current assignment scope, but it also means that BattleEngine is the most complex class in the system and could be further decomposed in future iterations.

## 1.2 Understanding the Problem and Requirements

### 1.2.1 Core Features

Following the assignment requirements, the system was designed to implement a turn-based combat game in which a single player battles one or more enemies through a sequence of rounds. During each round, every combatant acts exactly once in an order determined by their speed stat. Beyond basic attack mechanics, the system must also support non-trivial gameplay rules such as consumable items, temporary and permanent status effects, special-skill cooldowns, and backup enemy waves. These requirements motivated the introduction of dedicated abstractions such as TurnOrderStrategy, StatusEffect, and SkillCooldown, allowing the game rules to remain modular and extensible rather than being embedded directly within BattleEngine.

The key functionalities include:

| Component | Description |
|---|---|
| Combat System | A round-based battle between one player and multiple enemies. The battle proceeds until either the player dies or all enemies, including backup enemies, are defeated. |
| Turn Order | Each round, combatants are sorted in descending order of speed through SpeedTurnOrder, which implements TurnOrderStrategy. |
| Player Actions | On each player turn, the player may choose one of four actions: BasicAttack, Defend, UseItem, or SpecialSkill. |
| Enemy Behaviour | Enemies automatically execute BasicAttack on the player each turn |
| HP Management | HP is stored per combatant and clamped within [0, maxHP]. Damage, healing, and defeat are all processed through changeStat(). |
| Item Usage | Two single-use items selected at the start of the game; each item may be used only once and is removed from the inventory after use. |

| | |
|---|---|
| Status Effects | Status effects modify combat behaviour over time, with turn-based or round-based duration tracking |
| Backup Spawn | Additional enemies spawn when the initial wave is fully defeated, per level configuration |

The system ensures:

- Valid combat flow with no illegal moves, enforced through action validation in BattleEngine and cooldown checking in SkillCooldown
- Correct sequence of turns each round, determined by SpeedTurnOrder sorting combatants by speed in descending order
- Accurate HP and status effect updates, with changeStat() enforcing bounds and invulnerability checks, and updateTurnBasedEffects() managing effect lifecycles
- A guaranteed game ending, as the battle concludes either when the player's HP reaches zero or when all enemies including backup spawns are defeated

A particularly important design requirement was the distinction between turn-based and round-based effects. StunEffect prevents a combatant from acting and decreases after that combatant's turn. In contrast, DefendEffect and SmokeBombInvulnerabilityEffect remain active for an entire round and decrease only at round end. This difference motivated the design of separate update methods, namely updateTurnBasedEffects() and updateRoundBasedEffects(), rather than using a single generic tick method.

### 1.2.2 Additional Features

- Two player classes
  - Warrior (HP: 260, ATK: 40, DEF: 20, SPD: 30) with Shield Bash, which deals BasicAttack damage and stuns the target for two turns
  - Wizard (HP: 200, ATK: 50, DEF: 10, SPD: 20) with Arcane Blast, which deals BasicAttack damage to all enemies and grants a permanent +10 ATK bonus for each kill until the end of the level
- Two enemy types

    - Goblin (HP: 55, ATK: 35, DEF: 15, SPD: 25)
    - Wolf (HP: 40, ATK: 45, DEF: 5, SPD: 35)
    - Each with different stat profiles that create varied combat scenarios
- Three difficulty levels
    - Easy (3 Goblins)
    - Medium (1 Goblin and 1 Wolf, with backup of 2 Wolves)
    - Hard (2 Goblins, with backup of 1 Goblin and 2 Wolves)
- Status effect system
    - StunEffect (prevents action for 2 turns, ticks per turn)
    - DefendEffect (adds +10 DEF for 2 rounds, ticks per round)
    - SmokeBombInvulnerabilityEffect (blocks all incoming damage for 2 rounds, ticks per round)
    - ArcaneBlastEffect (permanently increases ATK by 10 per kill, persists until end of level)
- Modular action system
    - Actions are encapsulated as subclasses of Action, allowing BasicAttack, Defend, UseItem, and SpecialSkill to be executed uniformly by BattleEngine without conditional logic
- Special skill cooldown system
    - SkillCooldown manages cooldown state independently from Player, tracking remaining turns, decrementing after each owner turn, and providing accurate display values both mid-round and at the end of a round.
- Three consumable item types
    - Potion (heals 100 HP, capped at max HP)
    - PowerStone (triggers the special skill effect once without affecting cooldown)
    - SmokeBomb (applies SmokeBombInvulnerabilityEffect for 2 rounds)
- Replay functionality
    - After each battle, the player can replay with the same settings, start a new game, or exit

### 1.2.3 Assumptions and Constraints

Assumptions:

- Only one player participates in combat at a time. The system is designed around a single Player instance per BattleEngine session.
- Combat strictly follows a turn-based structure. Each combatant takes exactly one action per round, in order of speed. No combatant may act out of turn.
- Each action is validated before execution. Special skill usage is gated by SkillCooldown.isAvailable(), item usage is only offered when items remain in the player's inventory, and stunned combatants are skipped without consuming a turn choice.
- HP values are bounded by each combatant's maximum. The Warrior has a maximum HP of 260, the Wizard 200, the Goblin 55, and the Wolf 40. These bounds are enforced through changeStat(), which clamps HP to [0, maxHP] on every modification.

Constraints:

- The game uses a Command Line Interface and does not include a graphical user interface. The game runs entirely in the terminal with text-based menus and output.
- No persistent data storage required. All game state exists in memory for the duration of a session and is not saved between runs.
- The current implementation assumes a relatively small number of combatants and therefore uses simple list traversal and sorting rather than more advanced data structures.

### 1.3 Application of Object-Oriented Concepts

- Encapsulation: Class attributes were restricted to prevent direct external access. The class provides controlled entry points for modification through public methods rather than allowing external classes to modify values directly. For example, the Combatant class manages health and status effects internally. Changes to the status can only be made through the method changeStat(StatType, int). Similarly, the statusEffects list is never exposed directly. External classes must call on dedicated methods such as applyEffect(), updateTurnBasedEffects(), updateRoundBasedEffects, and clearAllEffects() to make changes to it.

- Inheritance: Inheritance is used extensively to reduce code duplication. Combatant serves as the base class for both Player and Enemy, defining shared state and behaviour such as

HP management, stat changes, and status effect tracking. Player extends Combatant to add inventory management through addItem() and removeItem(), special skill handling via getSpecialSkill(), and cooldown tracking via SkillCooldown. Warrior and Wizard then extend Player, contributing only their unique special skill by passing a ShieldBash or ArcaneBlast instance to the Player constructor, with no duplicated logic between them. The same pattern applies to StatusEffect, where DefendEffect, StunEffect, SmokeBombInvulnerabilityEffect, and ArcaneBlastEffect each override only the methods relevant to their behaviour, inheriting safe defaults for everything else.

- Polymorphism: Polymorphism allows different behaviours to execute depending on the object type at runtime. StatusEffect subclasses each override a different subset of methods. StunEffect overrides preventAction() and tickOnTurnEnd(), DefendEffect overrides onApply(), onRemove(), and tickOnRoundEnd(), and SmokeBombInvulnerabilityEffect overrides invulnerable() and tickOnRoundEnd(). Combatant calls these methods uniformly on every effect in statusEffects without any conditional logic to determine which effect type it is. Similarly, Action subclasses each implement execute() differently. BasicAttack applies damage, Defend applies a DefendEffect, and UseItem delegates to the item's use() method. BattleEngine calls execute() on any Action without knowing its concrete type, enabling new actions to be added without modifying the engine.

- Abstraction: Abstract classes define shared structure and contract without specifying implementation details. Action declares execute(Combatant actor, Combatant target, BattleEngine engine) as an abstract method, forcing all subclasses to provide their own implementation. StatusEffect defines the full lifecycle interface including onApply(), onTurnEnd(), onRoundEnd(), onRemove(), preventAction(), and invulnerable(), with safe default implementations that subclasses selectively override. This means Combatant can call effect.preventAction() or effect.invulnerable() on any effect without knowing its concrete type. SpecialSkill extends Action and adds applySkillEffect() as an abstract method, separating the cooldown management logic which is shared, from the skill-specific effect which differs between Warrior and Wizard. Item declares use() as abstract, allowing Potion, PowerStone, and SmokeBomb to each define their own behaviour when consumed.

## 1.4 Application of OOD Principles - SOLID

### 1.4.1 Single Responsibility Principle (SRP)

SRP is applied by giving each major class one primary area of responsibility, which improves maintainability and reduces the risk of unintended side effects when changes are made:

- Combatant manages combat state including HP, stats, and status effects.

- Player extends Combatant with inventory and skill cooldown management.
- SkillCooldown manages cooldown state independently, including turn reduction and display logic, separated from Player to keep cooldown concerns isolated.
- Each StatusEffect subclass encapsulates one effect's lifecycle behaviour.
- Each Action subclass encapsulates one unit of combat behaviour.
- GameCLI handles all input and output, and GameUI defines the interface contract between the engine and the UI layer.

While SRP is generally applied throughout the system, BattleEngine serves as the main coordination class and therefore takes on several closely related responsibilities, including turn sequencing, player interaction flow, enemy spawning, action execution, and battle-end checking. For the current assignment, consolidating these responsibilities within a single class simplified integration and made the overall control flow easier to follow. However, as the system grows, BattleEngine may become increasingly complex. A future enhancement could further improve maintainability by separating some of these responsibilities into smaller helper classes, such as a TurnProcessor or BattleStateChecker.

### 1.4.2 Open-Closed Principle (OCP)

The system is designed to be open for extension while minimising the need to modify existing code. Most new gameplay behaviour can be introduced through subclassing existing abstractions:

- New enemy types can be added by creating additional subclasses of Enemy.
- New status effects can be added by extending StatusEffect and overriding only the relevant lifecycle methods, without modifying Combatant or BattleEngine.
- New special skills can be added by extending SpecialSkill and implementing applySkillEffect(), with cooldown handling inherited automatically.
- New item types can be introduced through subclasses of Item.

Within the existing combat framework, new behaviours can therefore usually be added without changing the core battle logic. However, some parts of the system remain less extensible. For

example, introducing an entirely new category of player action would still require updates to the player-action selection flow in GameCLI and BattleEngine, since the available action options are currently defined explicitly.

### 1.4.3 Liskov Substitution Principle (LSP)

The system satisfies the Liskov Substitution Principle because subclasses preserve the behaviour expected of their parent classes and can therefore be used interchangeably without affecting correctness.

Player and Enemy are both subclasses of Combatant, and Warrior, Wizard, Goblin, and Wolf can all be treated uniformly as Combatant objects. BattleEngine stores and processes battle participants through Combatant references in methods such as runRound() and processTurn(). It calls methods such as isAlive(), canAct(), changeStat(), updateTurnBasedEffects(), and updateRoundBasedEffects() without needing to know the concrete subtype.

For example, when determining turn order, SpeedTurnOrder operates on a list of Combatant objects and can sort a mixture of Warrior, Goblin, and Wolf instances using only their shared getSpeed() behaviour. Likewise, BattleEngine can process a Goblin turn and a Warrior turn through the same control flow because both satisfy the contract defined by Combatant.

No subclass strengthens preconditions or weakens expected behaviour. For instance, all subclasses of Combatant still support HP modification, status-effect handling, and turn processing in the same way. As a result, new combatant subclasses can be introduced without requiring changes to code that already depends on Combatant.

### 1.4.4 Interface Segregation Principle (ISP)

The system applies ISP by ensuring that each interface or abstraction contains only the operations required by its clients.

TurnOrderStrategy defines only a single method, determineOrder(), because BattleEngine only needs to request the order in which combatants should act. Similarly, GameUI contains only methods related to user interaction, such as displaying messages and requesting player choices, without exposing any battle logic.

As a result, classes depend only on the behaviour they actually require. BattleEngine does not need to depend on the concrete details of GameCLI, and turn-order implementations do not need to implement unrelated functionality. This keeps the abstractions narrow, focused, and easier to extend.

### 1.4.5 Dependency Inversion Principle (DIP)

The system applies DIP by ensuring that higher-level modules depend on abstractions rather than concrete implementations.

BattleEngine depends on the TurnOrderStrategy abstraction rather than directly on SpeedTurnOrder. The concrete strategy is provided through the constructor, allowing different turn-order mechanisms to be introduced without modifying the battle engine. Similarly, BattleEngine depends on the GameUI abstraction rather than the concrete GameCLI class. This means the user interface could be replaced by a graphical or web-based implementation without changing the underlying battle logic. PowerStone also interacts with the player's special skill through the SpecialSkill abstraction rather than checking whether the player is a Warrior or Wizard. Consequently, additional player classes and special skills can be introduced without requiring changes to PowerStone.

By depending on abstractions rather than concrete classes, the system reduces coupling and makes future extensions easier to implement.

## 1.5 Trade-offs and Maintainability Considerations

One trade-off is in LevelFactory, where enemy instances are created directly using new Goblin() and new Wolf(). This means LevelFactory is tightly coupled to the concrete Goblin and Wolf

classes. If a new enemy type were introduced, LevelFactory would need to be modified directly, which is a violation of OCP. A more extensible approach would be to define enemy configurations externally and instantiate them dynamically, so LevelFactory would not need to change when new enemy types are added.

A second trade-off is in removeDeadEnemies() in BattleEngine, which permanently removes defeated enemies from activeEnemies using removeIf(). While this keeps the active enemy list clean, it means eliminated enemies cannot be referenced later. For example, displaying a full post-battle summary of all enemies defeated throughout the game would not be possible without additional tracking. Maintaining a separate list of all enemies alongside activeEnemies would have preserved this information, at the cost of slightly more state to manage.

A third trade-off is in Main, where player recreation on replay is handled by checking originalPlayer.getName() and using string comparison to determine which player class to instantiate. This is fragile because it relies on the player's name matching exactly. If the name of Warrior or Wizard were changed, the replay logic would silently break. A more maintainable approach would be to store the player class or use a factory, removing the dependency on string matching entirely.

Lastly, BattleEngine is also a central coordination class and currently contains logic for turn sequencing, player input flow, action execution, enemy spawning, effect updates, and win/loss detection. Consolidating these responsibilities simplified implementation and made the overall battle flow easier to follow for the current assignment. However, it also increases coupling and means that many future changes are likely to require modifications to BattleEngine. As the system grows, this may reduce maintainability and make the class harder to test. A more extensible design could separate these responsibilities into helper classes such as TurnProcessor, EnemySpawner, and BattleStateChecker, reducing the complexity of the engine and bringing it more fully in line with SRP.

### 1.6 Design Patterns Used

The system incorporates several design patterns:

- Strategy Pattern

  TurnOrderStrategy is a direct application of the Strategy pattern. BattleEngine accepts a TurnOrderStrategy in its constructor and calls determineOrder() each round. SpeedTurnOrder implements this by sorting combatants using Comparator.comparingInt(Combatant::getSpeed).reversed(). A different strategy such as a random initiative roll could be introduced by implementing the interface and passing it into BattleEngine at construction, with no changes to existing code.

- Template Method Pattern

  SpecialSkill defines the template in execute(), which handles cooldown checking and activation, then delegates to the abstract applySkillEffect() for the subclass to implement. ShieldBash applies damage and a StunEffect to a single target, while ArcaneBlast iterates over all alive enemies, applies damage to each, and grants an ArcaneBlastEffect to the actor for each kill. Both inherit the cooldown logic entirely without duplicating it. The same applies to StatusEffect, where the tick and lifecycle structure is defined in the base class and subclasses override only what they need.

- Command-like Action Encapsulation

  Each Action subclass encapsulates a complete, self-contained unit of behaviour. BattleEngine calls execute() without knowing which action it is, decoupling action selection in GameCLI from action execution in BattleEngine. UseItem further demonstrates this by wrapping an Item inside an Action, so the engine treats item usage identically to any other action without special-casing it.

## 2. Detailed UML Class Diagram

Please refer to Figures 2.1a–2.1d for the detailed UML class diagrams.

The system's class structure is organized according to the Boundary–Control–Entity (BCE) architectural pattern, which separates user interaction, battle coordination, and game data into distinct layers. To improve readability, the UML is not presented as a single monolithic diagram. Instead, it is divided into four complementary views, each focusing on a different aspect of the system.

| Diagram | Description |
|---|---|
| Figure 2.1a: High-Level System Overview | Provides an overview of the overall architecture and the relationships between the major classes across all BCE layers. This diagram serves as the entry point for understanding how the system is structured before examining each layer in greater detail. |
| Figure 2.1b: Boundary Layer | Focuses on GameUI and GameCLI, which are responsible for all user interaction. This layer encapsulates menu display, player input, action selection, and battle result presentation, while remaining independent of the underlying battle logic. BattleEngine depends only on the GameUI abstraction rather than directly on GameCLI. |
| Figure 2.1c: Control Layer | Details BattleEngine and its supporting classes, including TurnOrderStrategy, SpeedTurnOrder, Level, LevelFactory, |

| | |
|---|---|
| | BattleResult, and Difficulty. Together, these classes coordinate turn sequencing, enemy spawning, round progression, and battle termination. The layer demonstrates the use of the Strategy pattern through TurnOrderStrategy. |
| Figure 2.1d: Entity Layer | Captures the core game entities and gameplay rules. This includes Combatant, Player, Enemy, Action, Item, SpecialSkill, StatusEffect, and their respective subclasses. This layer models the state and behaviour of the game independently of how it is displayed or controlled, allowing the combat logic to remain reusable and extensible. |

Please refer to <UML Diagrams/High-Level System Overview Diagram>, <UML Diagrams/Boundary UML>, <UML Diagrams/Controller UML>, and <UML Diagrams/Entity UML> in the GitHub repository for the high-definition versions of the diagrams.

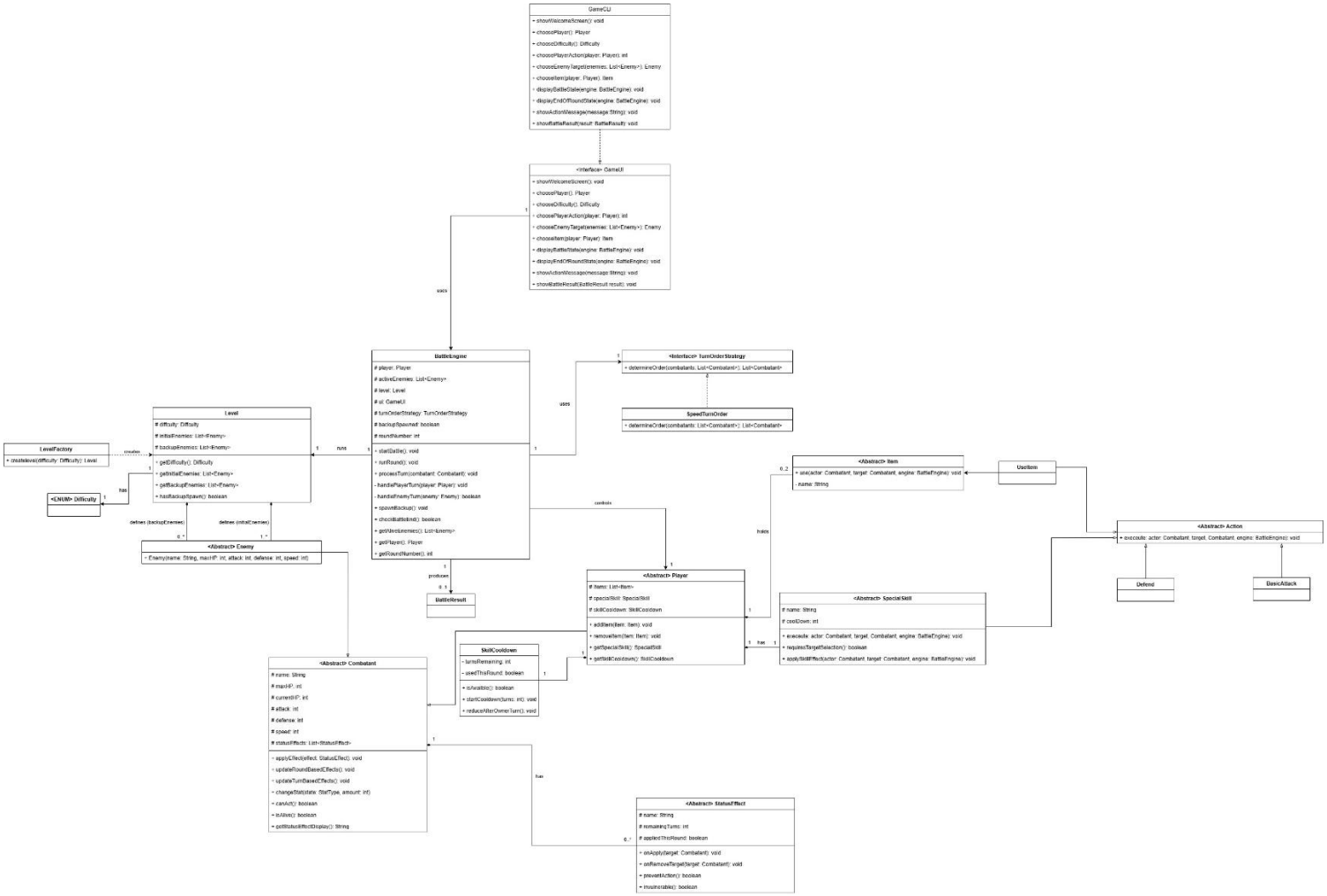


Figure 2.1a: High-Level System Overview Diagram

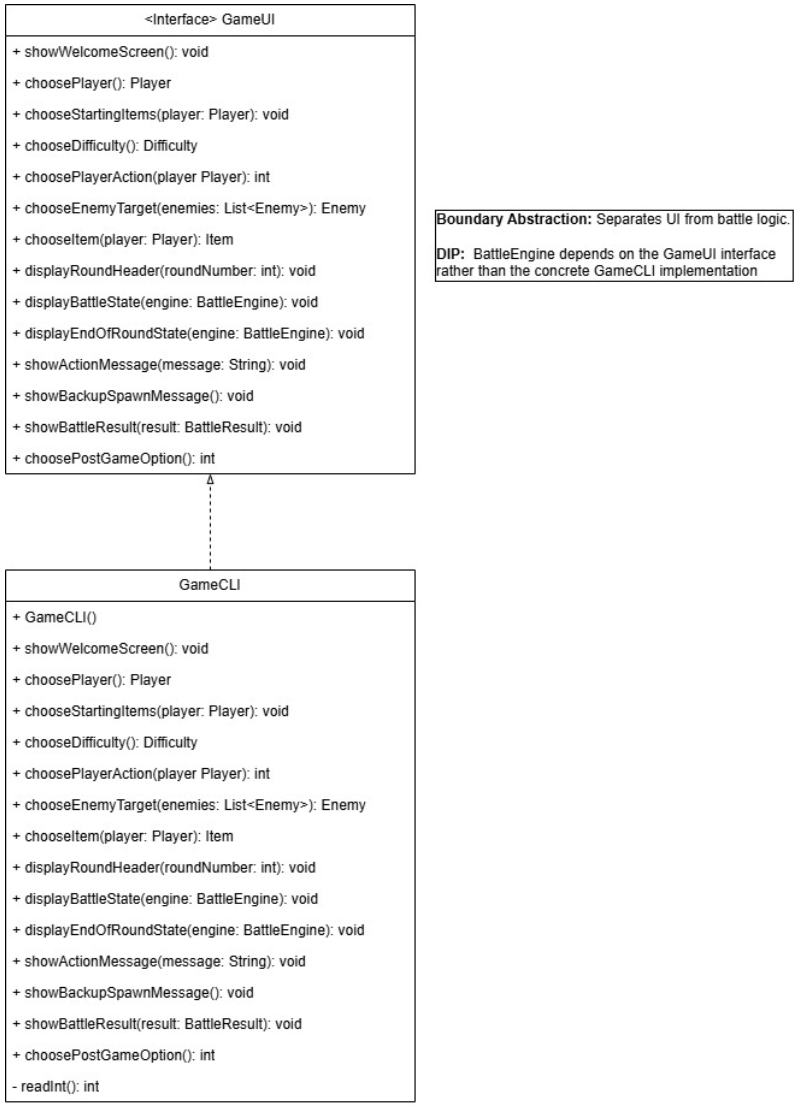


Figure 2.1b: Boundary Layer

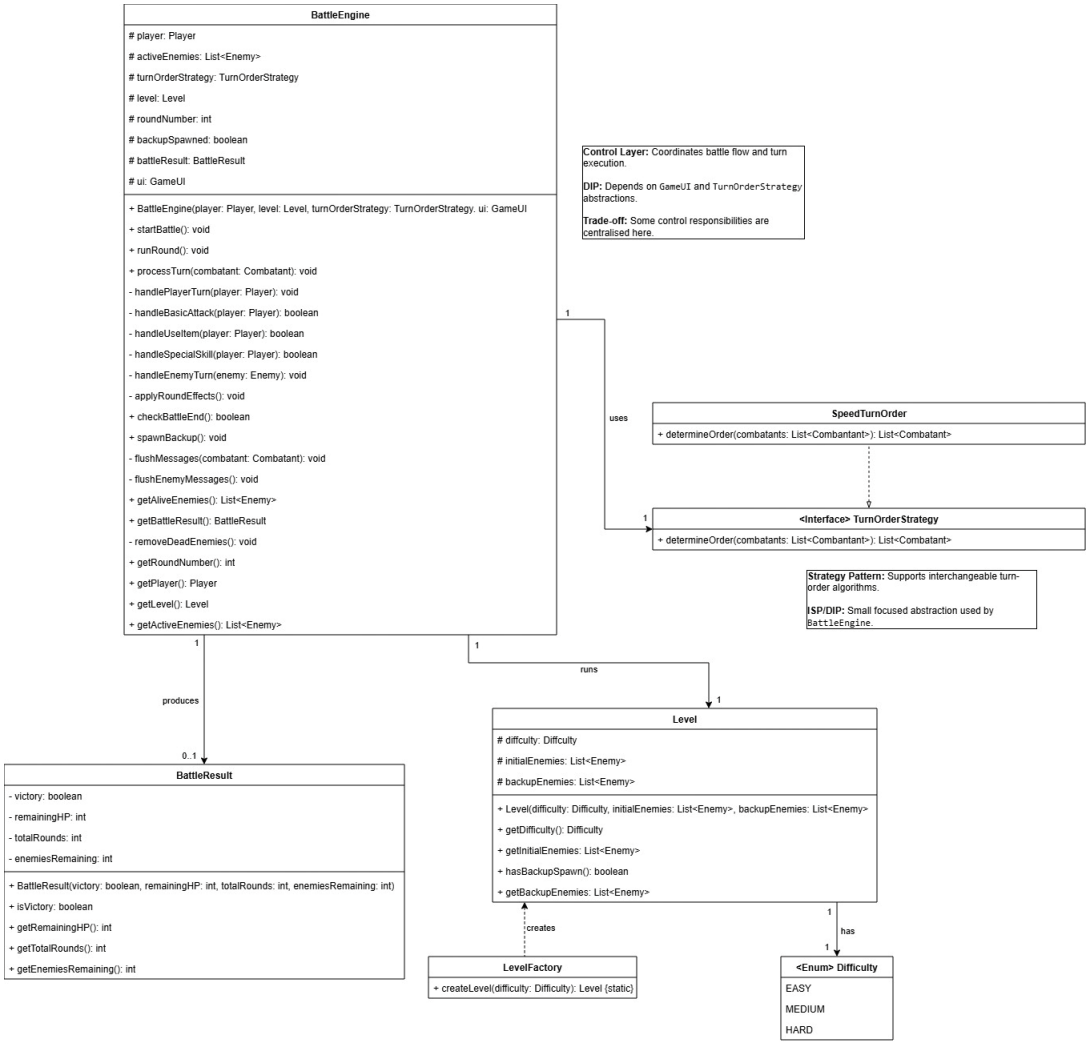


Figure 2.1c: Control Layer

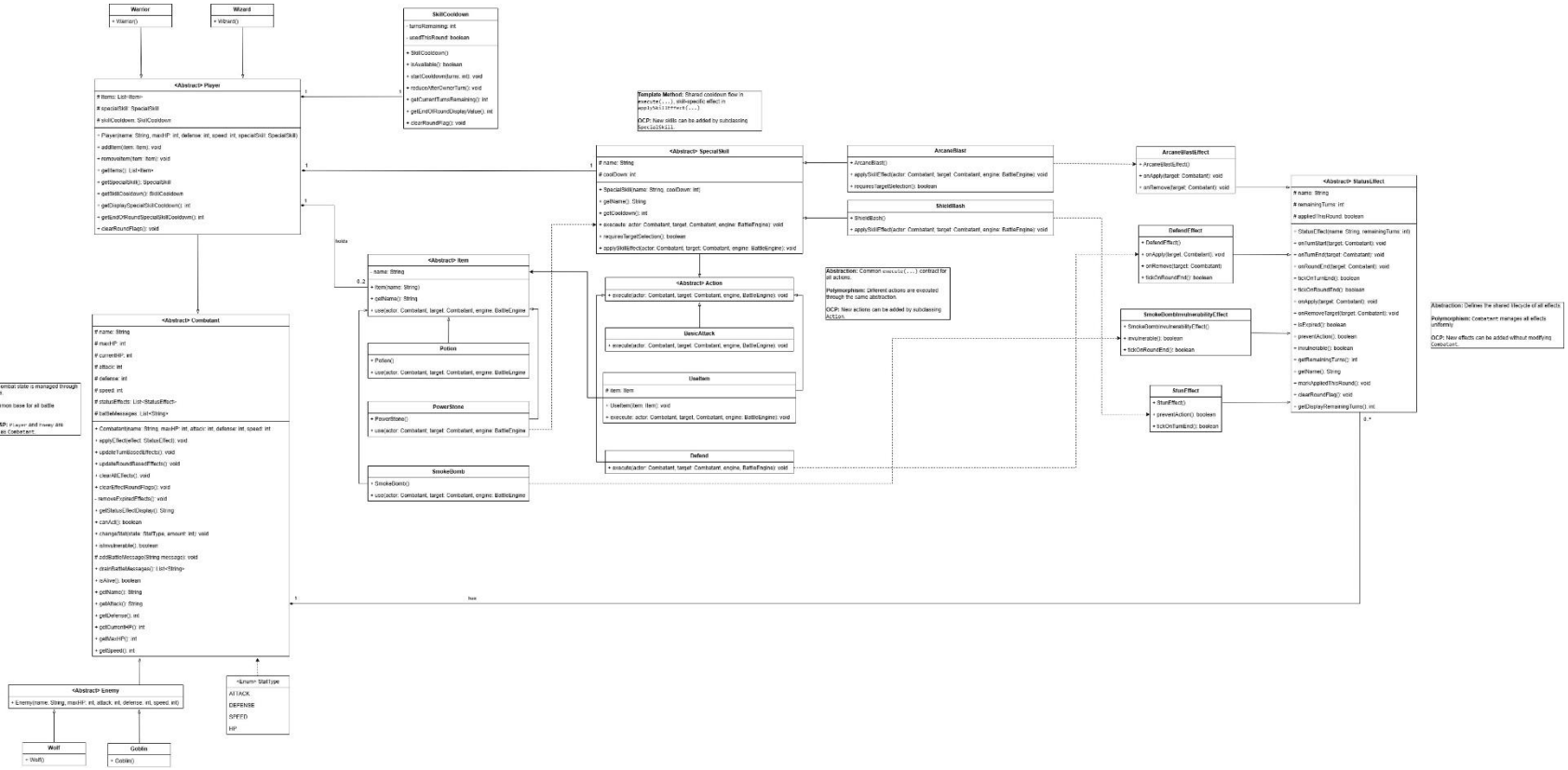


Figure 2.1d: Entity Layer

## 3. UML Sequence Diagram

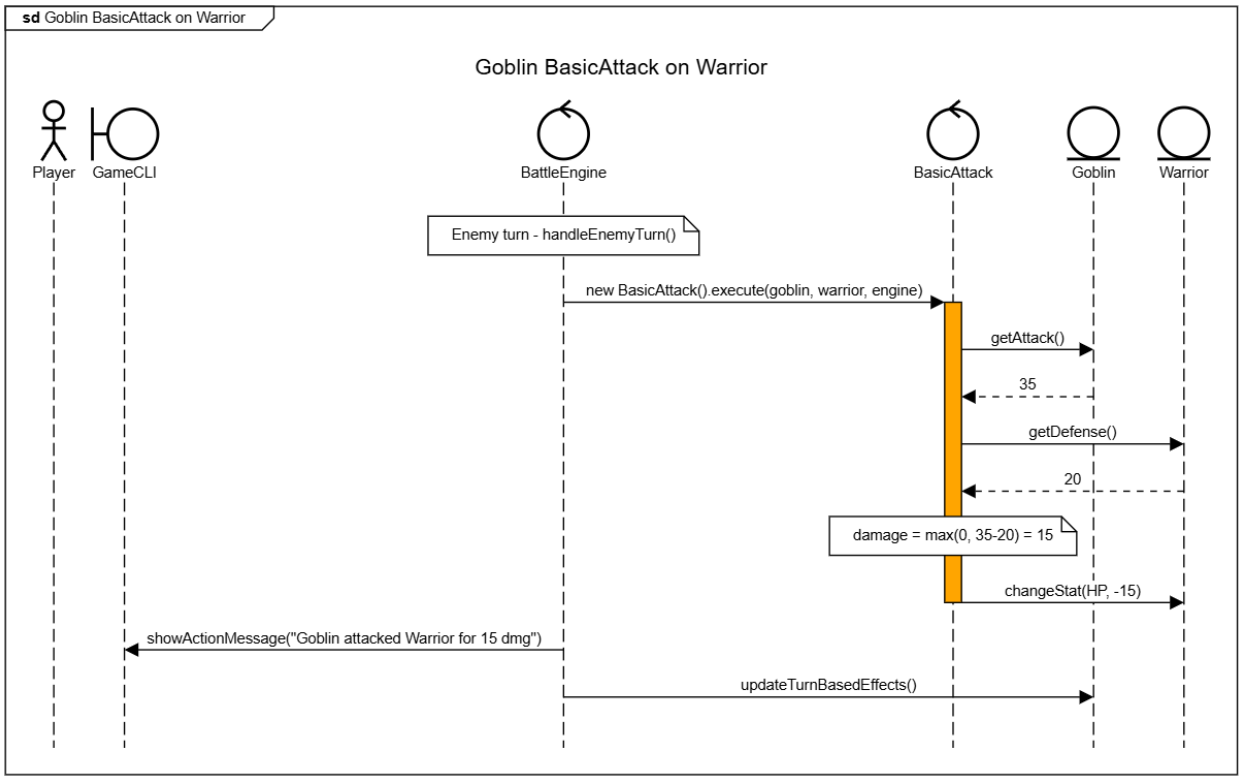


Fig 3.1. Sequence Diagram showing Goblin using Basic Attack on Warrior

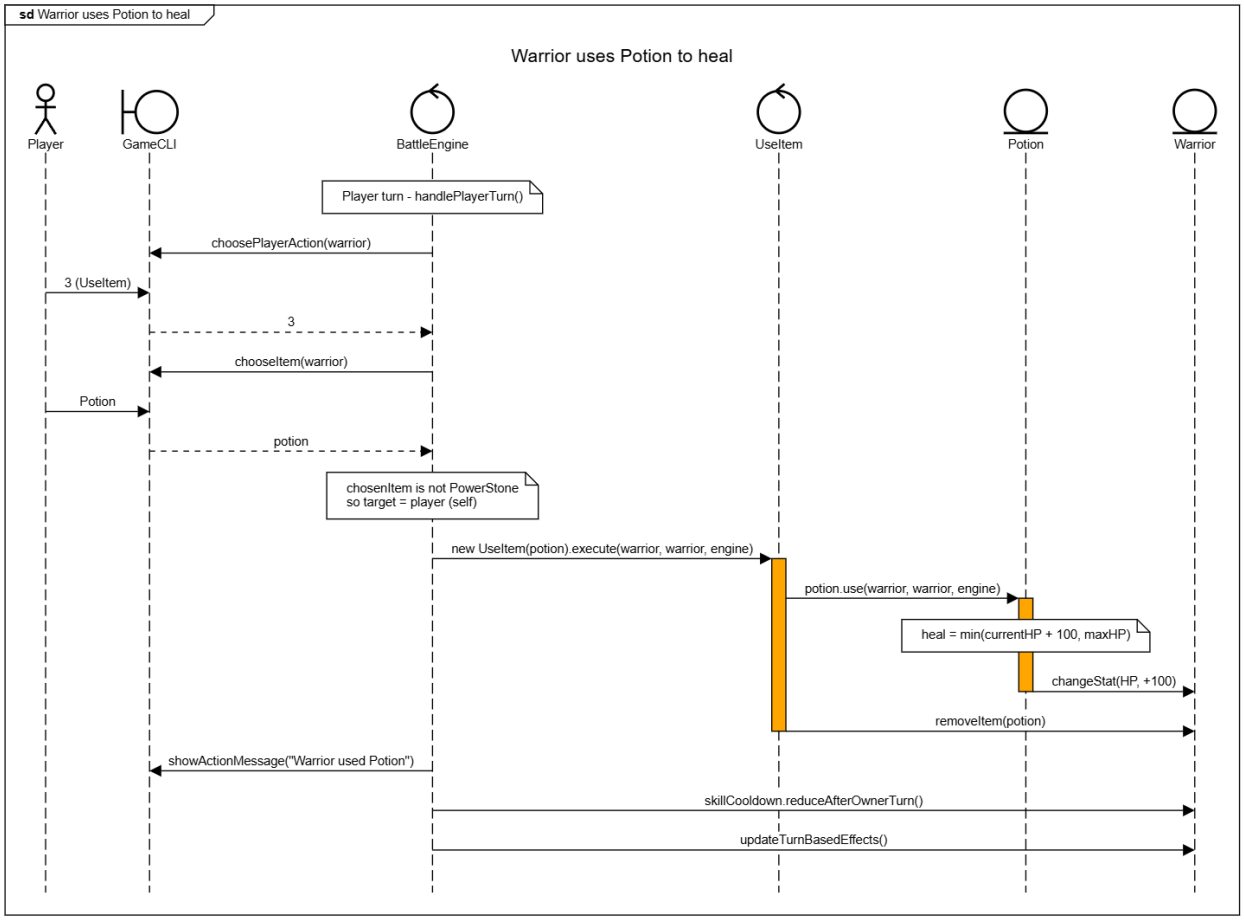


Fig 3.2. Sequence Diagram showing Warrior using Potion to Heal

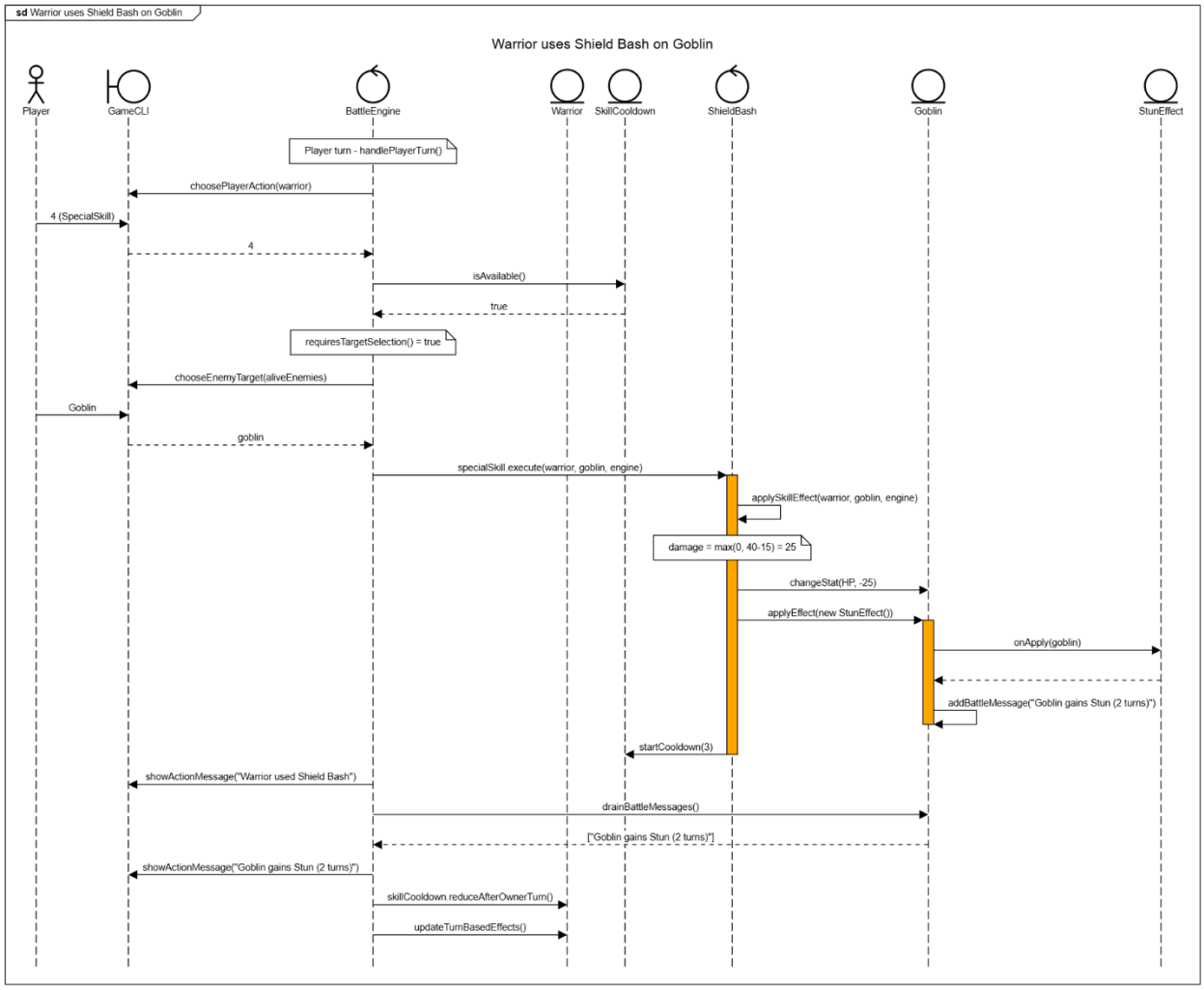


Fig 3.3. Sequence Diagram showing Warrior using Shield Bash on Goblin

## 4. Testing

We manually tested our system through all normal, boundary and edge cases. Through multiple rounds of testing, we ensured that our system fits all requirements, with the necessary inputs. To test our system, we start by running the main class Main.java.

**a.** Player and Game Setup

| **Flow** | **Edge/Test Cases** | **Sample User Input** |
|---|---|---|
| Choosing player: 1 for Warrior or 2 for Wizard | Edge: Invalid input (entering 0, 3, text etc.) | Enter choice: **1** |

| | | |
|---|---|---|
| Select starting items | Edge: Invalid input (entering 0, 4, text etc.) | Choose item 1: **1**<br>Choose item 2: **3** |
| Choose difficulty: | Edge: Invalid input (entering, 0,4, text etc.) | Enter choice: **2** |

**b.** Basic Combat Actions

| **Flow** | **Edge/Test Cases** | **Sample User Input** |
|---|---|---|
| Basic Attack | Edge: Invalid input of target (entering 0, 3, text etc.) | Enter choice: **1** |
| Defend | Test: Check if defend effect can be used consecutively | Enter choice: **2** |
| Use item | Edge: No items left<br>Edge: Invalid item index (entering 0, 3, text etc.) | Enter choice: **1** |

**c.** Special Skills

| **Flow** | **Edge/Test Cases** | **Sample User Input** |
|---|---|---|
| Use special skill | Edge: Invalid item index (entering 0, 3, text etc.) | Enter choice: **1** |
| Cooldown handling | Edge: Attempt to use skill on cooldown<br>Test: Using skill while on cooldown<br>Test: Check for cooldown count decreasing | Using skill after cooldown<br>Enter choice: **1** (after cooldown reaches 0) |

**d.** Item Effects

| **Flow** | **Edge/Test Cases** | **Sample User Input** |
|---|---|---|
| Potion | Edge: HP exceeds maxHP<br>Test: Check if Potion can be used when HP is maxed | Using Potion when HP is less than maxHP |

| | | |
|---|---|---|
| Smoke Bomb | Test: Effect duration is handled correctly | *No user input required* |
| Power Stone | Test: Power Stone activates as intended | *No user input required* |

**e.** Status Effects

| **Flow** | **Edge/Test Cases** | **Sample User Input** |
|---|---|---|
| Stun Effect | Test: Stun not preventing action | *No user input required* |
| Effect Expiry | Test: Effect is removed after duration ends | *No user input required* |
| Permanent Effects | Test: Permanent buffs remain as intended | *No user input required* |

**f.** Enemy and Turn System

| **Flow** | **Edge/Test Cases** | **Sample User Input** |
|---|---|---|
| Turn Order | Test: Faster entity always acts first | *No user input required* |
| Enemy Attack | Test: Correct calculation of damage | *No user input required* |

**g.** Battle Flow

| **Flow** | **Edge/Test Cases** | **Sample User Input** |
|---|---|---|
| Backup Spawn | Test: Correct backup spawn | *No user input required* |
| Victory Condition | Test: Battle ends when all enemies are defeated | *No user input required* |
| Defeat Condition | Test: Battle ends when Player HP reaches 0 | *No user input required* |

**h.** Post-game Interface

| **Flow** | **Edge/Test Cases** | **Sample User Input** |
|---|---|---|
| Replay Game | Test: State is restored properly | Enter choice: **1** |
| Restart Game | Test: State of game is reset | Enter choice: **2** |
| Exit Game | Test: Game terminates | Enter choice: **3** |

## 5. Reflection

### 5.1 Development Challenges

One of the earliest difficulties we encountered was designing the status effect system. Initially, we were unsure how to handle effects that tick on different schedules. Some effects such as StunEffect needed to decrease per turn, while others such as DefendEffect and SmokeBombInvulnerabilityEffect needed to decrease per round. Our first approach used a single tick method applied uniformly to all effects, which caused incorrect durations. We resolved this by introducing tickOnTurnEnd() and tickOnRoundEnd() as separate boolean methods in StatusEffect, allowing each subclass to declare which schedule it follows. This taught us the value of designing for flexibility early, rather than retrofitting behaviour later.

Handling the PowerStone item also presented a challenge. The PowerStone needed to trigger the player's special skill effect without starting or modifying the cooldown. Since SpecialSkill.execute() always starts the cooldown, we could not call it directly. We resolved this by having PowerStone.use() call applySkillEffect() directly, bypassing the cooldown management in execute(). While this works correctly, it does expose internal implementation details of SpecialSkill to PowerStone, which is a coupling we noted as a limitation.

Coordinating code across the group also proved to be a challenge. As different members worked on different classes simultaneously, we encountered situations where changes made by one member would break code written by another, particularly in the early stages before our class interfaces were fully agreed upon. For example, changes to method signatures in Combatant or BattleEngine would require updates across multiple files owned by different members, leading to merge conflicts and integration errors. We overcame this by establishing the core interfaces and abstract classes first, namely Action, Item, StatusEffect, and GameUI, before dividing implementation work. This ensured that each member had a clear contract to code against, and that individual components could be developed and tested independently before being integrated. This experience reinforced the practical value of designing abstractions before implementations, which is a principle we had studied in lectures but only fully appreciated through the difficulties of group development.

Additionally, for simplicity and to avoid further merge conflicts during testing, we consolidated our workflow by coding in Visual Studio Code and designating one member to review and push all finalized code into the shared GitHub repository, ensuring that the codebase remained consistent and conflict-free throughout the project.

### 5.2 Takeaways

This assignment provided practical experience applying SOLID principles in a non-trivial system. Prior to this, concepts such as DIP and ISP were understood theoretically but difficult to motivate. Designing BattleEngine to depend on TurnOrderStrategy and GameUI as interfaces rather than concrete classes made the benefit of DIP tangible as we could change the turn order mechanism or swap the UI without touching the engine. Similarly, the OCP became meaningful when we added ArcaneBlastEffect and SmokeBombInvulnerabilityEffect without modifying Combatant or BattleEngine at all.

We also gained a deeper understanding of the Template Method pattern through SpecialSkill. Separating the invariant cooldown logic in execute() from the variant skill behaviour in applySkillEffect() meant that adding a new special skill required no consideration of cooldown mechanics at all. This demonstrated concretely how the pattern reduces the cognitive load of extending a system.

Working as a group also taught us the importance of agreeing on interfaces and abstractions before implementing concrete classes. Early in the project, parallel development caused integration issues when method signatures did not match. Defining GameUI, TurnOrderStrategy, and Action as contracts first allowed team members to work independently with confidence that their code would integrate correctly.

### 5.3 Further Improvements

The player recreation logic in Main relies on name string matching to determine which player class to reinstantiate on replay. This could be replaced with a player factory or by storing a reference to a creation function, making the replay logic robust to name changes and easier to extend when new player types are added.

Finally, as the game grows, BattleEngine may benefit from being split into smaller components. A dedicated TurnProcessor could handle the logic for a single combatant's turn, and a BattleStateChecker could handle win and loss condition evaluation. This would bring BattleEngine more fully in line with SRP and make each component independently testable.